SMART EV CHARGING ALLOCATOR

AI-based EV charging station allocation using CSP

Presented by: [Srinith] · [24R11a6675]



THE PROBLEM

Charging becomes a scheduling problem

Multiple EVs may request charging at the same time, while stations have limited availability, power, and compatibility.



Different requirements

Different EVs have different battery levels, charging needs, arrival times, and departure deadlines.



Allocation conflicts

Poor allocation creates conflicts, idle chargers, and unnecessary waiting.



Software simulation

This project simulates an intelligent charging allocator in software, without physical EV hardware.



AI-assisted schedule

AI search and constraint-based reasoning generate a valid charging schedule.

PROJECT TARGETS

Objectives

01

Build

Build an intelligent EV charging allocation system.

02

Model

Model charging as a Constraint Satisfaction Problem.

03

Assign

Assign EVs to suitable stations and time slots.

04

Prevent

Avoid station, time, and compatibility conflicts.

05

Prioritize

Prioritize EVs using Greedy Search.

06

Repair

Use Backtracking when an assignment violates constraints.

07

Optimize

Optimize waiting time, cost, and station utilization.

08

Connect

Provide an interactive web interface.

SYSTEM ARCHITECTURE

From web request to final schedule

1. USER
2. HTML + CSS INTERFACE
3. FLASK BACKEND
4. PYTHON AI ENGINE
5. GREEDY SEARCH
6. CSP + BACKTRACKING
7. OPTIMIZATION
8. FINAL EV CHARGING SCHEDULE

INPUTS

Battery level
Arrival/departure time
Charging requirement
Station availability
Station power

OUTPUTS

Assigned station
Charging start/end time
Allocation status
Constraint/conflict status

CONSTRAINT SATISFACTION PROBLEM

EV → Station → Time Slot

Model each allocation as a Constraint Satisfaction Problem: variables represent EVs, domains contain compatible stations and available time slots, and constraints remove invalid combinations.

EV

Station

Time Slot

VALID ONLY IF

- A station cannot charge conflicting EVs simultaneously.
- The EV must arrive before charging begins and finish before departure.
- The station must be available and powerful enough for the requirement.
- Charging need and charger compatibility must be satisfied.

PRIORITIZATION

Greedy Search sets the order

At each step, select the EV with the strongest immediate need and assign the most suitable available option.

01

Priority signals

Earliest departure, lowest battery level, highest urgency, and limited charging opportunity.

02

Immediate choice

Select the strongest immediate need and the most suitable available option.

03

Fast ordering

Greedy Search is fast and practical for prioritization.

EXAMPLE ORDER

EV2 → Highest Priority
EV1 → Medium Priority
EV3 → Lower Priority

Important: Greedy Search does not guarantee a globally optimal solution.

CONFLICT REPAIR

Backtracking repairs conflicts

When a choice breaks a constraint, return to the earlier decision and try another valid assignment.

1 EV1 → Station 1

2 EV2 → Station 1

3 Conflict detected

Overlapping use of Station 1.



BACKTRACK

VALID ALLOCATION

EV2 → Station 2

Valid allocation found.

Optimize the valid schedules

After finding feasible allocations, compare them and select the schedule with the best overall score.

MINIMIZE

Total Waiting Time + Charging Cost

Subject to all CSP constraints being satisfied.



Waiting time

Minimize waiting time.



Charging cost

Reduce charging cost.



Utilization

Improve station utilization.



Urgency

Respect EV urgency.



Idle time

Avoid unnecessary idle charging time.

IMPLEMENTATION MAP

Code description

01

Flask setup

Creates the backend server and connects the web interface with Python.

02

Station data

Stores station name, type, power, and availability.

03

EV data

Stores battery level, arrival/departure time, and charging requirement.

04

Greedy priority

Calculates the order in which EVs are considered.

05

CSP validation

Checks station and time-slot constraints.

06

Backtracking

Reverts invalid assignments and tries alternatives.

07

Optimization

Compares valid schedules using waiting time, cost, and utilization.

08

Final allocation

Returns the selected station and schedule.

09

API endpoint

Receives frontend data and returns allocation results.

10

Web execution

Flask runs the application locally.

REVIEW CHECKPOINT

Three implementation challenges



Constraint conflicts

- Multiple EVs may request one station.
- Overlapping charging times create conflicts.
- Charger compatibility must be maintained.



Scheduling

- Arrival and departure windows must be respected.
- Charging duration must fit the available time.
- Prioritization must handle urgent EVs.

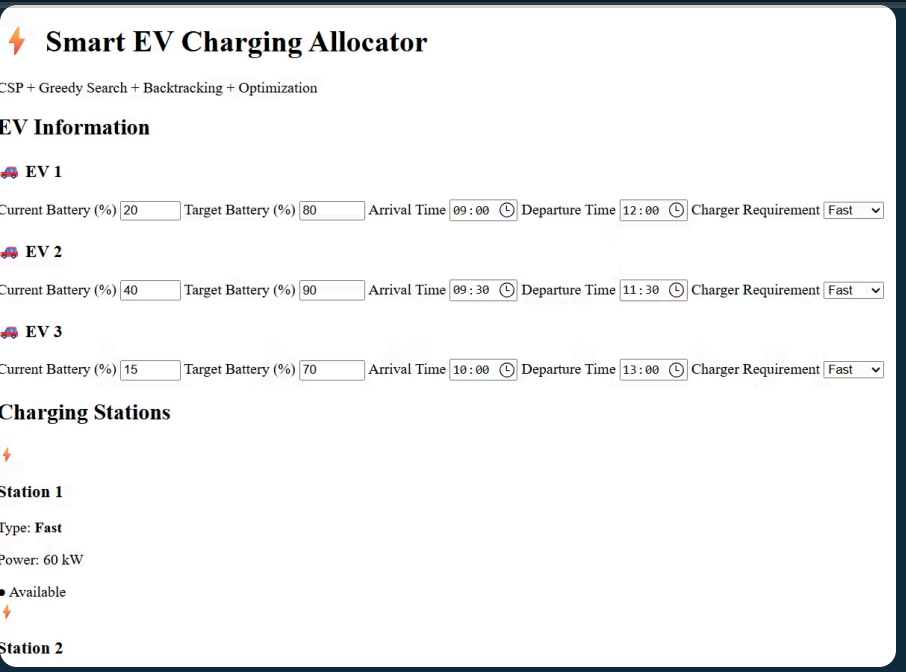


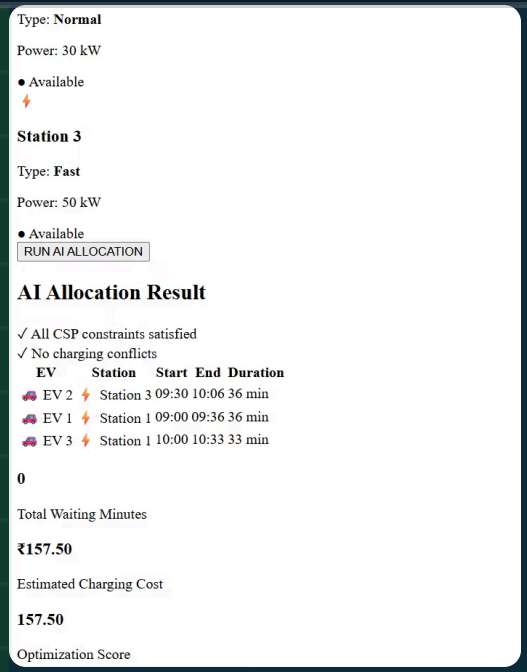
Data communication

- Frontend and Flask must exchange correct data.
- Missing or incorrect information can cause allocation errors.

Working Prototype

EV Inputs, Charging Stations & AI Allocation Result





THANK YOU